自由制作課題（グループ）

内山 航平(25A4111026), 今 蒼月(25A4111068)

目次

- 1. Main ソースコード(内山 担当)
- 1.1. メインロジック
- 2. PreparationJp(日本語対応) (今 担当)
- 2.1. Preparationを継承した日本語用のデータ準備クラス
- 3. Probability (内山 担当)
- 4. Learning (内山 担当)
- 4.1. HashMapを用いた単語予測
- 5. Generate (内山 担当)
- 5.1. 再帰関数を用いたリアルタイム出力
- 5.2. Generate 結果
- ソースコードについて

1. Main ソースコード(内山 担当)



```
Console  Preparation.jp.java  Main.java  Probability.java  Preparation.java  Generate.java  Learning.java
package markov_chain_practice;

import java.io.IOException;

public class Main {
    public static void main(String[] args) {
        // Scannerと全体でのエラーハンドリングのためtry
        try {
            var sc = new Scanner(System.in);
            var is_jp = false;
            // var use_custom = false;

            // datasetの選択。数字以外(parse to intに失敗)だった場合はパスとして取得
            System.out.println();
            System.out.println("[STEP 1] Dataset Selection");
            System.out.println("----------------------------------------------------");
            System.out.println("Chose a dataset type:");

            // 各数字とデフォルトのセットを対応
            System.out.println("  0: [ALL] \t - Combine all available data");
            System.out.println("  1: [NORMAL] \t - Starndard data set");
            System.out.println("  2: [SF] \t - SF dataset");
            System.out.println("  3: [SHIKA] \t - shikanoko special dataset");
            System.out.println("  4: [NORMAL2] \t - Nature and Environment");
            System.out.println("  [CUSTOM] \t - custom dataset path (numbers prohibited)");
            System.out.printf("\nSelect number (or input path)> ");

            var input = sc.nextLine();

            // 入力値がintに変換出来ればデフォルトのデータセットを選択
            // 失敗すれば入力をそのままパスとして取り込む
            // "\\d+"は数値にマッチする。マッチしなければ真ではない
            var is_numric = input.matches("\\d+");
            // 数字でなければパスが直接入力されたということ
            var use_custom = !is_numric;

            // ?:'演算子を用いて、`cond ? 数字の場合の処理 : パス入力の場合`
            List<Path> lst = is_numric ? (switch (Integer.parseInt(input)) {
                case 1 -> List.of(Path.of(default_para().toString(), "normal.txt"));
                case 2 -> List.of(Path.of(default_para().toString(), "sf.txt"));
                case 3 -> List.of(Path.of(default_para().toString(), "shika.txt"));
                case 4 -> List.of(Path.of(default_para().toString(), "normal2.txt"));
                default -> get_all_dataset_path();
            }) : List.of(Path.of(input));

            // カスタムソースのパスの場合は言語推定が出来てない(悪しい)のでユーザーに質問
            if (use_custom) {
                System.out.printf("is Your source Japanese?(y/n)>");
                if (sc.next().trim() == "y") {
                    is_jp = true;
                }
            }

            System.out.printf("\n----------------------------------------------------\n");
            System.out.printf("[INFO] Loading dataset...\n");

            // Preparationにてデータの取得と単語へのパースを行う
            var preparation_data = is_jp ? new PreparationJp(lst.toArray(Path[]::new)) : new PreparationEn(lst.toArray(Path[]::new));

            // ここ以降はPreparation super classで抽象化してあるから変更不要
            var parsed_data = preparation_data.Parse(); // 単語に分割したメソッドになる

            var loaded_num = parsed_data.length;
            var loaded_path_lst = preparation_data.LoadedPathList();
```

```
Preparation.jp.java  Main.java
            System.out.printf("[PROCESS]\n");
            System.out.printf("  - Analyzing loaded text...");
            // Leaningにて各単語の出現回数をカウントしてベクトル的に格納
            var leaned_data = new Learning(parsed_data);
            System.out.printf(" [DONE]\n");

            // Generateにて出現回数から次単語の出現確率を各ベクトルに変換
            System.out.printf("  - Calculating probabilities...");
            var caled = new Generate(leaned_data);
            System.out.printf(" [DONE]\n");

            System.out.println();
            System.out.println("----------------------------------------------------");
            System.out.println("Markov Chain Model is ready!");
            System.out.println("----------------------------------------------------");

            var file_name_lst = IntStream.range(0, loaded_path_lst.size())
                    .mapToObj(i -> {
                        var spaces = (i == 0) ? "" : "              ";
                        return String.format("%s%s\n", spaces, loaded_path_lst.get(i).getFileName().toString());
                    })
                    .collect(Collectors.joining());
            System.out.println();
            // 何単語分生成するかを質問
            System.out.printf("[CONFIG]\n  - Dataset: %s\nHow long to generate> ", file_name_lst);
            var i = sc.nextInt();
            sc.nextLine(); // 空文字を消費して後に残さない

            System.out.printf("Seed word (\"***\" or not found = random)> ");
            var in_seed = sc.nextLine();
            String seed;
            if ((in_seed.isEmpty()) || in_seed.equals("***") || (!caled.checkExistWord(in_seed))) {
                seed = caled.randomStart();
            } else {
                seed = in_seed;
            }

            System.out.printf("How many generations?>");
            var in_num = sc.nextLine();
            var nn = parse_option(in_num)
                    .map(n -> {
                        return n;
                    })
                    .orElse(1);
            sc.close();

            for (int j = 1; j < nn + 1; j++) {
                System.out.printf("\n[RESULT]\n----------------------------------------------------\n\n");
                // iは生成する単語数
                var gened_words_count = caled.loopShow(seed, i, 0);
                System.out.printf("\n----------------------------------------------------\n");

                System.out.printf(
                        "\n[STATUS]\n  - Words generated: %d\n  - Count: %d\n  - Status: Success\n  - Seed word: \"%s\"\n",
                        gened_words_count, j, seed);
            }

            System.exit(0);
        } catch (Exception e) {
            System.err.printf("\n\n[ERROR] \n%s\n", e);
            System.exit(1);
        }
    }
```

1.1. メインロジック

- 主に `Preparation` , `Leaning` , `Generate` を用いてクラスにデータを変換していく
- それぞれのクラスは単一の責任を持つように設計し、処理の差し替えを容易化
- 全体的にCUIでも読みやすいように空行を用いたり `---` などの記号で可読性を向上

2. PreparationJp(日本語対応) (今 担当)

PreparationJp.java

```java
package markov_chain_practice;

import java.nio.file.Path;

public class PreparationJp extends Preparation {

    public PreparationJp(Path[] p) throws Exception {

        super(p);
    }

    public String[] Parse() {
        Tokenizer tokenizer = new Tokenizer();
        String text = this.input;

        // 分割してList<String>に変換
        List<String> words = tokenizer.tokenize(text).stream()
                .map(Token::getSurface)
                .collect(Collectors.toList());

        return words.toArray(String[]::new);
    }

}
```

2.1. Preparationを継承した日本語用のデータ準備クラス

- `Parse()` メソッドをオーバーライドすることで日本語特有の単語分割に処理を置き換え
- それ以外の処理はすべて親クラスに依存
- Kkuromoji libを利用することで意味的な日本語の単語分割を実現

3. Probability (内山 担当)

```java

package markov_chain_practice;

public class Probability {
    // 分子
    long num;
    // 分母
    long den;

    /**
     *
     * @param n: 分子
     * @param d: 分母
     */
    public Probability(long n, long d) {
        this.num = n;
        this.den = d;
    }

    // ゲッター
    public long get_num() {
        return this.num;
    }

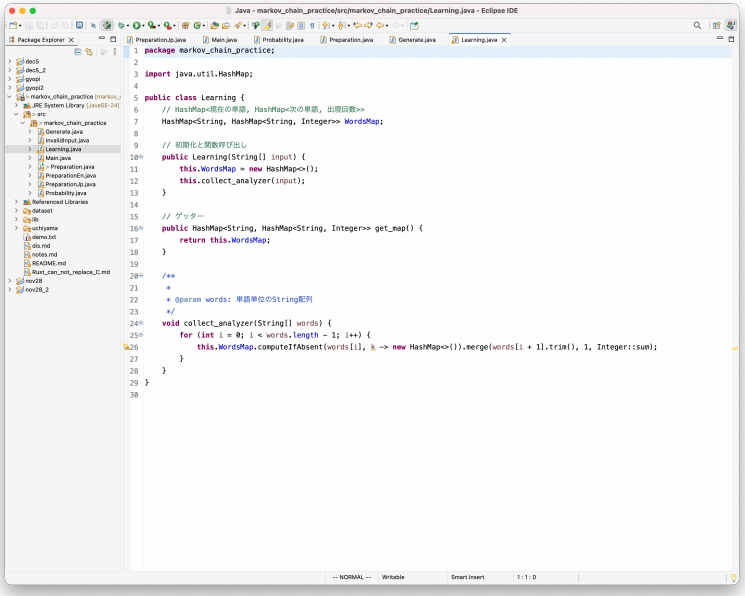
    public long get_den() {
        return this.den;
    }

    // Debug用に表示系を
    @Override
    public String toString() {
        return String.format("%d/%d", this.num, this.den);
    }
}

```

- 確率情報を分数として保持するためのクラス
- 小数点数で保持すると精度の問題で厳密でなくなるため、分数として保持することに

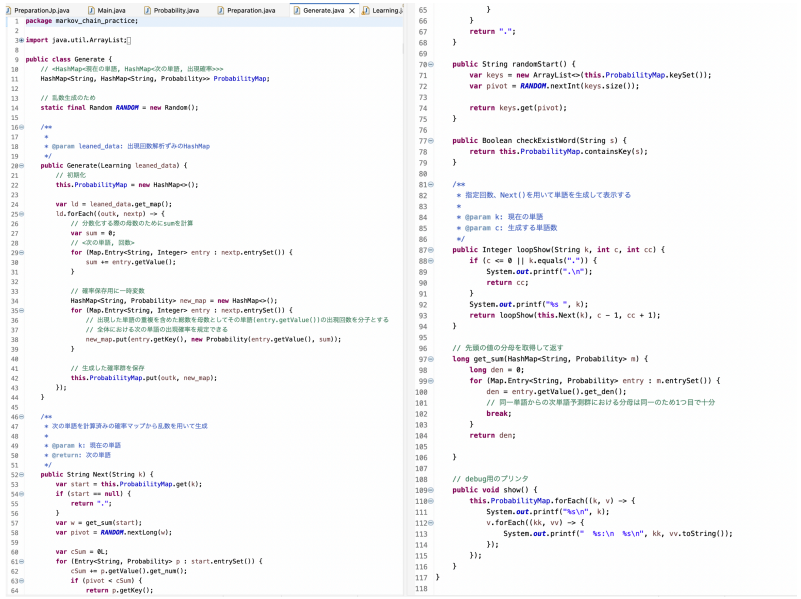
4. Learning (内山 担当)



4.1. HashMapを用いた単語予測

- `HashMap<現在の単語, HashMap<次の単語, 出現回数>>` となるようにカウントを行う
- メソッドを用いることで単語の出現確率演算をシンプルに記述

5. Generate (内山 担当)



```java
package markov_chain_practice;

import java.util.ArrayList;

public class Generate {
    // <HashMap<現在の単語, HashMap<次の単語, 出現確率>>>
    HashMap<String, HashMap<String, Probability>> ProbabilityMap;

    // 乱数生成のため
    static final Random RANDOM = new Random();

    /**
     *
     * @param leaned_data: 出現回数解析ずみのHashMap
     */
    public Generate(Learning leaned_data) {
        // 初期化
        this.ProbabilityMap = new HashMap<>();

        var ld = leaned_data.get_map();
        ld.forEach((outk, nextp) -> {
            // 分数化する際の母数のためにsumを計算
            var sum = 0;
            // <次の単語, 回数>
            for (Map.Entry<String, Integer> entry : nextp.entrySet()) {
                sum += entry.getValue();
            }

            // 確率保存用に一時変数
            HashMap<String, Probability> new_map = new HashMap<>();
            for (Map.Entry<String, Integer> entry : nextp.entrySet()) {
                // 出現した単語の重複を含めた総数を母数としてその単語(entry.getValue())の出現回数を分子とする
                // 全体に占める次の単語の出現確率を規定できる
                new_map.put(entry.getKey(), new Probability(entry.getValue(), sum));
            }

            // 生成した確率群を保存
            this.ProbabilityMap.put(outk, new_map);
        });
    }

    /**
     * 次の単語を計算済みの確率マップから乱数を用いて生成
     *
     * @param k: 現在の単語
     * @return: 次の単語
     */
    public String Next(String k) {
        var start = this.ProbabilityMap.get(k);
        if (start == null) {
            return ".";
        }
        var w = get_sum(start);
        var pivot = RANDOM.nextLong(w);

        var cSum = 0L;
        for (Entry<String, Probability> p : start.entrySet()) {
            cSum += p.getValue().get_num();
            if (pivot < cSum) {
                return p.getKey();
            }
        }
        return ".";
    }

    public String randomStart() {
        var keys = new ArrayList<>(this.ProbabilityMap.keySet());
        var pivot = RANDOM.nextInt(keys.size());

        return keys.get(pivot);
    }

    public Boolean checkExistWord(String s) {
        return this.ProbabilityMap.containsKey(s);
    }

    /**
     * 指定回数、Next()を用いて単語を生成して表示する
     *
     * @param k: 現在の単語
     * @param c: 生成する単語数
     */
    public Integer loopShow(String k, int c, int cc) {
        if (c <= 0 || k.equals(".")) {
            System.out.printf(".\n");
            return cc;
        }
        System.out.printf("%s ", k);
        return loopShow(this.Next(k), c - 1, cc + 1);
    }

    // 先頭の値の分母を取得して返す
    long get_sum(HashMap<String, Probability> m) {
        long den = 0;
        for (Map.Entry<String, Probability> entry : m.entrySet()) {
            den = entry.getValue().get_den();
            // 同一単語からの次単語予測群における分母は同一のため1つ目で十分
            break;
        }
        return den;

    }

    // debug用のプリンタ
    public void show() {
        this.ProbabilityMap.forEach((k, v) -> {
            System.out.printf("%s\n", k);
            v.forEach((kk, vv) -> {
                System.out.printf("  %s:\n  %s\n", kk, vv.toString());
            });
        });
    }
}
```

5.1. 再帰関数を用いたリアルタイム出力

- 再帰しながら常に出力を行うことでリアルタイムに出力を行う

5.2. Generate 結果

```
[CONFIG]
  - Dataset: shika.txt

How long to generate> 14
Seed word ("***" or not found = random)> し
How many generations?>

[RESULT]
----------------------------------------

し た ん た ん た ん た ん た ん た ん た .

----------------------------------------

[STATUS]
  - Words generated: 14
```

ソースコードについて

現在githubのprivate repositoryにソースコード全体がアップロードされています。
uliboulibo@gmail.comにgithubアカウントのidと共にご連絡頂ければ、こちらでrepositoryに招待し、ソースコードを閲覧できるようにいたします。